

**Московский государственный технический
университет им. Н. Э. Баумана**

Факультет «Информатика и системы управления»
Кафедра ИУ5 «Системы обработки информации и управления»

Курс «Парадигмы и конструкции языков программирования»
Отчет по рубежному контролю №1
«Вариант А, 17»

Выполнила:
Студент группы ИУ5-31Б
Сербенюк Полина
Проверил:
Гапанюк Ю. Е.

2025 г.

Classes.py

```
class Conductor:
```

```
    """Дирижер"""
```

```
    def __init__(self, id, name, experience, orchestra_id):
```

```
        self.id = id
```

```
        self.name = name
```

```
        self.experience = experience # стаж работы в годах
```

```
(количественный признак)
```

```
        self.orchestra_id = orchestra_id
```

```
class Orchestra:
```

```
    """Оркестр"""
```

```
    def __init__(self, id, name):
```

```
        self.id = id
```

```
        self.name = name
```

```
class ConductorOrchestra:
```

```
    """
```

```
'Дирижеры оркестра' для реализации
```

```
связи многие-ко-многим
```

```
"""
```

```
    def __init__(self, orchestra_id, conductor_id):
```

```
        self.orchestra_id = orchestra_id
```

```
        self.conductor_id = conductor_id
```

data.py

```
from classes import Conductor, Orchestra, ConductorOrchestra
```

```
orchestras = [
```

```
    Orchestra(1, 'Большой симфонический оркестр'),
```

```
    Orchestra(2, 'Камерный оркестр "Виртуозы")',
```

```
    Orchestra(3, 'Симфонический оркестр филармонии'),
```

```
    Orchestra(4, 'Студийный оркестр'),
```

```
    Orchestra(5, 'Эстрадный оркестр'),
```

```
]
```

Дирижеры

```
conductors = [  
    Conductor(1, 'Светланов', 40, 1),  
    Conductor(2, 'Плетнев', 35, 2),  
    Conductor(3, 'Федосеев', 45, 3),  
    Conductor(4, 'Гергиев', 38, 3),  
    Conductor(5, 'Баршай', 30, 4),  
    Conductor(6, 'Китаенко', 42, 1),  
    Conductor(7, 'Синайский', 36, 5),  
]
```

```
conductors_orchestras = [  
    ConductorOrchestra(1, 1),  
    ConductorOrchestra(1, 6),  
    ConductorOrchestra(2, 2),  
    ConductorOrchestra(3, 3),  
    ConductorOrchestra(3, 4),  
    ConductorOrchestra(4, 5),  
    ConductorOrchestra(5, 7),
```

Дополнительные связи для многие-ко-многим

```
    ConductorOrchestra(1, 3), # Светланов также дирижирует  
Симфоническим оркестром
```

```
    ConductorOrchestra(2, 7), # Плетнев также дирижирует  
Эстрадным оркестром
```

```
    ConductorOrchestra(3, 1), # Федосеев также дирижирует  
Большим симфоническим
```

```
    ConductorOrchestra(4, 2), # Баршай также дирижирует  
Камерным оркестром
```

```
    ConductorOrchestra(5, 5), # Синайский также дирижирует  
Студийным оркестром
```

```
]
```

requests.py

```
from operator import itemgetter  
import data
```

```
def create_one_to_many():
```

```
    """Создание соединения один-ко-многим"""
```

```
    return [(c.name, c.experience, o.name)
```

```
            for o in data.orchestras
```

```
            for c in data.conductors
```

```
            if c.orchestra_id == o.id]
```

```
def create_many_to_many():
```

```
    """Создание соединения многие-ко-многим"""
```

```
    many_to_many_temp = [(o.name, co.orchestra_id,  
co.conductor_id)
```

```
                        for o in data.orchestras
```

```
                        for co in data.conductors_orchestras
```

```
                        if o.id == co.orchestra_id]
```

```
    return [(c.name, c.experience, orch_name)
```

```
            for orch_name, orch_id, cond_id in many_to_many_temp
```

```
            for c in data.conductors if c.id == cond_id]
```

```
def request_1(one_to_many):
```

```
    """
```

```
    Задание А1
```

```
    «Оркестр» и «Дирижер» связаны соотношением один-ко-  
многим.
```

```
    Выведите список всех связанных дирижеров и оркестров,  
отсортированный по оркестрам, сортировка по  
дирижерам произвольная.
```

```
    """
```

```
    print('Задание А1')
```

```
    print('Список всех дирижеров и оркестров, отсортированный  
по оркестрам:')
```

```
    res_11 = sorted(one_to_many, key=itemgetter(2)) # сортировка  
по названию оркестра
```

```
    for item in res_11:
```

```
    print(f' {item[0]} - {item[1]} лет ({item[2]})')
print()
return res_11
```

```
def request_2(one_to_many):
```

```
    """
```

Задание A2

«Оркестр» и «Дирижер» связаны соотношением один-ко-многом.

Выведите список оркестров с суммарным стажем дирижеров в каждом оркестре, отсортированный по суммарному стажу.

```
    """
```

```
    print('Задание A2')
    print('Список оркестров с суммарным стажем дирижеров, отсортированный по суммарному стажу:')

    # Используем словарь для группировки по оркестрам
    orchestra_experience = {}
    for conductor_name, experience, orchestra_name in
one_to_many:
```

```
        if orchestra_name not in orchestra_experience:
            orchestra_experience[orchestra_name] = 0
            orchestra_experience[orchestra_name] += experience
```

```
    # Сортировка по суммарному стажу (по убыванию)
    res_12 = sorted(orchestra_experience.items(), key=itemgetter(1),
reverse=True)
```

```
    for orchestra_name, total_experience in res_12:
        print(f' {orchestra_name}: {total_experience} лет')
    print()
    return res_12
```

```
def request_3(many_to_many):
```

"""

Задание А3

«Оркестр» и «Дирижер» связаны соотношением многие-ко-многим.

Выведите список всех оркестров, у которых в названии присутствует слово «оркестр»,
и список работающих в них дирижеров.

"""

```
print('Задание А3')
print(
    'Список всех оркестров, у которых в названии присутствует
    слово "оркестр", и список работающих в них дирижеров:')

# Используем словарь для группировки дирижеров по
оркестрам
res_13 = {}

# Фильтруем оркестры, содержащие "оркестр" в названии
filtered_orchestras = [o for o in data.orchestras if 'оркестр' in
o.name.lower()]

for orchestra in filtered_orchestras:
    # Находим всех дирижеров для данного оркестра
    orchestra_conductors = [conductor_name
                             for conductor_name, experience, orch_name in
many_to_many
                             if orch_name == orchestra.name]
    # Убираем дубликаты
    unique_conductors = list(set(orchestra_conductors))
    if unique_conductors:
        res_13[orchestra.name] = unique_conductors

for orchestra_name, conductors_list in res_13.items():
    print(f' {orchestra_name}:')
    for conductor in conductors_list:
        print(f' - {conductor}')
print()
```

```
return res_13
```

```
def main():  
    """Основная функция для выполнения всех запросов"""  
    # Создаем соединения  
    one_to_many = create_one_to_many()  
    many_to_many = create_many_to_many()  
  
    # Выполняем запросы  
    request_1(one_to_many)  
    request_2(one_to_many)  
    request_3(many_to_many)  
  
if __name__ == '__main__':  
    main()
```

main.py

```
from requests import main  
  
if __name__ == '__main__':  
    main()
```